Hardening Change Document

MCP for Unity — security hardening pass, all changes

Repository	skevin220/unity-mcp
Landed on	beta (fast-forward dccecd6 → 3a66299), pushed
Work branch	harden/security (9 commits)
Scope	Server/ (Python) + MCPForUnity/Editor (C#) + CustomTools/ + tests/docs
Net change	27 files · +588 / –1776 lines (excludes the analysis PDF)
Date	2026-06-19

Purpose. This document records every change made in the security hardening pass, mapped to the audit findings (R1–R12). The goal was not to remove the Editor's code-execution capability — `manage_script` needs it — but to narrow the trust boundary from “any local process” to “any process that holds the token,” remove the worst always-on code-exec path, and ensure the audited source is the source that runs. The human approval gate in the MCP client remains the final backstop.

1. Commit summary

Commit	Finding	Summary	Δ
`b2c7549`	**R1**	Remove the runtime_compilation tool (CustomTools/RoslynRuntimeCompilation)	`+2 / −1744`
`d1c03e5`	**R1**	Disable the Roslyn DLL auto-installer (unverified NuGet fetch-and-run)	`+18`
`2e993b6`	**R4,R5**	Token-gate every local transport (stdio bridge, /api/command, WS hub)	`+319 / −2`
`58a67ce`	**R9**	Make tool enable/disable a real dispatch boundary	`+25 / −3`
`bfdcbd6`	**R6**	Default-disable code-exec tools; allow-list execute_menu_item	`+88 / −6`
`1f591a9`	**R11**	Pin the launched server to exact mcpforunityserver==9.7.3	`+59 / −16`
`1dd633d`	**R10**	Disable telemetry by default	`+10 / −5`
`c39b925`	**R8**	Fail loud on off-loopback HTTP binds	`+25`
`3a66299`	**docs**	Record hardening pass + accepted risks (R3, R12)	`+42`

Verification note: Python logic was validated directly (token resolver, constant-time gate, framed-auth roundtrip, telemetry-off, off-loopback decision matrix). The full pytest suite and a Unity compile could not be run in the build sandbox (deps/Editor unavailable); C# changes are verified by inspection and existing tests were read to confirm they stay green. A real compile + test run is recommended before release.

2. Changes in detail

R1 Remove the Roslyn runtime-compilation tool

What & why. The highest-exposure finding: a built-in, default-enabled tool whose `execute_with_roslyn` / `compile_and_load` actions compiled arbitrary C#, loaded it via `Assembly.Load`, added components and invoked methods inside the Editor — with no safety checks.

How. Deleted the entire `CustomTools/RoslynRuntimeCompilation/` directory so the tool cannot register; a direct socket call to `runtime_compilation` now returns "unknown tool." Separately, `RoslynInstaller.Install()` early-returns instead of downloading Microsoft.CodeAnalysis DLLs from NuGet (an unverified fetch-and-run path, R11); `execute_code` falls back to the built-in CodeDom (C# 6) compiler. Two stale installer strings that advertised the removed tool were corrected.

Verification. No external code referenced the deleted types (verified repo-wide). Installer was only invoked from two manual UI buttons, never on a clean start.

Files: CustomTools/RoslynRuntimeCompilation/ManageRuntimeCompilation.cs (+.meta), RoslynRuntimeCompiler.cs (+.meta) — deleted; MCPForUnity/Editor/Setup/RoslynInstaller.cs

---

R4 · R5 Token-gate the local bridge

What & why. The stdio TCP bridge (127.0.0.1:6400) and HTTP-local control plane (`/api/command`, WS hub) accepted any local connection with no identity check, and a new stdio connection could displace the live one (hijack). A shared secret converts the boundary to "any process that knows the token."

How. Token resolution is identical on both sides: `UNITY_MCP_BRIDGE_TOKEN` env wins, else a 0600 `~/.unity-mcp/bridge-token` the Editor generates. C#: the bridge requires the token as the first framed message *before* the stale-client close (so an unauthenticated peer cannot hijack), validated constant-time via `BridgeAuth`. HTTP-local: `/api/command` and the WS hub require an `X-Bridge-Token` header (constant-time, fail-closed); the Unity WS client and the CLI present it. Remote-hosted API-key auth is unchanged. Token files are git-ignored.

Verification. Functional test passed: resolver (env + file precedence), the hmac accept/reject gate, and the framed auth-frame roundtrip.

Files: NEW MCPForUnity/Editor/Helpers/BridgeAuth.cs (+.meta); StdioBridgeHost.cs; WebSocketTransportClient.cs; AuthConstants.cs; Server/src/transport/legacy/unity_connection.py; plugin_hub.py; main.py; cli/utils/connection.py; core/constants.py; .gitignore

---

R9 Make tool enable/disable a real dispatch boundary

What & why. The dispatcher already refused disabled tools on dispatch, but the default-enabled computation enabled every built-in regardless of group (`AutoRegister || IsBuiltIn`). That let a client speaking directly to the socket reach non-core tools (e.g. `execute_code`, group scripting_ext) the Python server would never advertise.

How. Centralized the default into `DefaultEnabledFor()`: `AutoRegister` still enables, but built-ins default-enable *only* for the "core" group. Non-core groups stay disabled until explicitly toggled, so the existing `IsToolEnabled` gate now rejects them by default — mirroring the server-side advertise policy.

Verification. Inspection only (no Unity compile in sandbox); the registry test checks registration, not enablement, so it stays green.

Files: MCPForUnity/Editor/Services/ToolDiscoveryService.cs

R6 Default-disable code-exec tools; allow-list execute_menu_item

What & why. Reduce the default-enabled surface to scene/object authoring. `execute_menu_item` previously denied only `File/Quit` — builds, asset deletion, package ops and third-party menus were all allowed.

How. `execute_code` is already off via R9. Added `execute_menu_item` to a default-disabled set (off until explicitly enabled). Play-mode entry (`manage_editor action=play`) now requires an opt-in EditorPref (`AllowPlayMode`,

default false), since entering play runs project code including scripts the AI just authored. `ExecuteMenuItem` replaced its deny-list with a conservative allow-list (GameObject/, Component/, Assets/Create/, Window/, select Edit/* items, File/Save) and rejects everything else.

Verification. Added allow-list accept/reject tests. Existing ExecuteMenuItem tests stay green (File/Quit still rejected; Window/ and File/Save still pass).

Files: EditorPrefKeys.cs; ToolDiscoveryService.cs; ExecuteMenuItem.cs; ManageEditor.cs; TestProjects/.../ExecuteMenuItemTests.cs

---

R11 Pin the launched server to an exact reviewed version

What & why. `uvx` fetched the server from PyPI with a floating spec — “mcpforunityserver” (latest) when the version was unknown, or the open prerelease range “mcpforunityserver>=0.0.0a0” for beta. A newer/compromised PyPI release would run unreviewed.

How. `GetMcpServerPackageSource()` now returns an exact pin (`mcpforunityserver==9.7.3`, matching the reviewed Server/pyproject.toml) via the documented `PinnedServerVersion` constant. A local Server Source Override still wins for development. The pin and re-pin procedure are documented.

Verification. Existing config-helper test invariants preserved (--from present, package ref intact, --prerelease only-if-present).

Files: MCPForUnity/Editor/Helpers/AssetPathUtility.cs; NEW docs/security/server-pin.md

---

R10 Disable telemetry by default

What & why. Anonymous opt-out telemetry to api-prod.coplay.dev can carry truncated Unity error strings that may embed file/asset paths.

How. `ServerConfig.telemetry_enabled` is now `False`, and `TelemetryConfig` treats a missing config as disabled too (defense-in-depth). The DISABLE_* env opt-outs still work; set `telemetry_enabled=True` to opt back in. The characterization test was updated to the new default.

Verification. Functional test passed: TelemetryConfig().enabled is False on a normal session.

Files: Server/src/core/config.py; core/telemetry.py; tests/test_core_infrastructure_characterization.py

---

R8 Fail loud on off-loopback HTTP binds

What & why. The bind host is operator-controllable and the shipped docker-compose binds 0.0.0.0, so one mis-set flag turned the unauthenticated local control plane into remote RCE with no warning.

How. Added a startup guard: a non-loopback host logs a prominent warning and, in local mode, refuses to start unless the bridge-token gate is active (a token is resolvable). Remote-hosted binds are allowed (API-key auth applies). Loopback remains the default.

Verification. Functional test passed: decision matrix (loopback ok / remote warn / local+token warn / local no-token refuse).

Files: Server/src/main.py

---

3. Accepted risks (documented, not changed)

Two findings were intentionally left in place because removing them would break required functionality. They are recorded in `docs/security/hardening.md`.

Finding	Why kept	Compensating control
R3 — manage_script can execute code via [InitializeOnLoad]	Code execution can't be removed while keeping script authoring, which is a core feature.	Disposable, git-backed project; human approval on script-writing calls in the MCP client; play-mode is opt-in (R6).
R12 — no local session isolation	Acceptable for single-user use on one trusted machine; remote-hosted mode is still per-user scoped and fails closed.	Do not run on a shared / CI / multi-user host.

4. Recommended follow-ups before release

- Run a real Unity Editor compile across the supported version matrix (`tools/check-unity-versions.sh`) and the EditMode test suite, plus `cd Server & uv run pytest` — none could run in the build sandbox.
- Confirm the end-to-end Claude → Python → Unity path works with `UNITY_MCP_BRIDGE_TOKEN` set (or the generated token file present).
- Surface an “Allow Play Mode” toggle and a bridge-token status indicator in the MCP for Unity Advanced Settings UI (the prefs/back-end exist; UI wiring was out of scope).
- If Roslyn C# 12 support is needed, place verified Microsoft.CodeAnalysis DLLs in Assets/Plugins/Roslyn/ manually, or re-add the installer with hash/signature verification.
- Re-pin the server version deliberately on each reviewed release (see docs/security/server-pin.md).

How to operate safely (recap). Keep binds on loopback; run on a single-user trusted machine; keep code-exec tools disabled unless needed; work in a disposable git-backed project; require per-tool approval in your MCP client. Full rationale in the companion *Security Analysis* PDF and `docs/security/hardening.md`.